

拓扑排序应用软件个人任务及感想

一、个人工作内容

在本次拓扑排序应用软件项目中，我负责 C 的部分，即主要负责关系图可视化与图片导出模块，具体包括 GraphPanel、LayoutManager 和 ImageExporter 三个核心类，对应任务书中的 T-C1 到 T-C8 的任务。

我的核心工作是把算法层输出的图数据以直观、可交互的方式绘制出来，并支持图片导出。具体实现包括：

1. 分层布局算法

用 Kahn 算法的变体计算每个节点的拓扑层号，同层节点采用重心法排序以减少边交叉，再根据画布尺寸动态分配横纵坐标，保证 100 个节点仍能清晰展示、不重叠。

2. 环形布局算法

作为含环图或特殊场景的兜底方案，把所有节点均匀分布在圆周上，半径根据画布短边自适应。

3. 图形绘制

绘制圆角矩形节点（宽度随文本自适应）、带箭头的有向边、自环，支持环路径红色高亮、拓扑序 5 色轮换、选中节点黄色高亮三种高亮状态，并在节点右上角增加序号徽章，显示该节点在当前拓扑序中的位置。

4. 交互与视图控制

实现滚轮缩放、按钮缩放、拖拽平移、拖动节点微调、双击节点查看入度/出度/先修/后继信息，以及缩放比例显示。

5. PNG 导出

实现 exportPNG，忽略当前视图的缩放和平移，遍历所有节点计算包围盒，并为图例预留独立空间，保证无论用户如何缩放或平移，导出的始终是

一张完整、清晰、中文不乱码的关系图。

6. 图片导出工具

ImageExporter 封装了目录选择和带时间戳的文件命名，同秒重复导出自动加序号，同时在导出失败时给出中文提示。

在项目推进过程中，我全程参与了 5 次线上的小组会议，并每天在群聊中对齐接口、反馈问题、同步进度。我与其他成员保持了密切的沟通，很多问题都是在群聊里通过测试反馈、截图说明和讨论修改后共同解决的。

二、技术上的成长与收获

这次项目让我把所学的图算法、Swing 绘图、面向对象设计真正落地为一个可运行的软件，收获远超预期。

1. 布局算法从理论到工程实现

以前学拓扑排序时，我只关心“能不能得到一个正确的顺序”。这次做分层布局时，我才发现要把它变成可读的图形，还需要考虑很多工程问题：层号怎么算、同层节点怎么排序才能减少边交叉、节点宽度不同时怎么避免重叠、画布尺寸变化后怎么自适应。通过实现 Kahn 变体算层号和重心法排序，我对“算法要服务于实际场景”有了更具体的体会。

2. Swing 绘图与视图变换

GraphPanel 是一个基于 Swing 的绘图组件，其中的节点、边、箭头、图例和缩放指示器都是通过绘图代码实现的。我还实现了以鼠标为中心的滚轮缩放和平移，理解了 Graphics2D 的 translate 和 scale 变换，以及如何把屏幕坐标反算回模型坐标来做节点命中检测。这些内容看起来简单但是在实际写代码的时候才发现需要注意的细节非常多。

3. 从“能画出来”到“能稳定导出”

PNG 导出是我这次遇到问题最多也成长最快的部分。最初版本在平移或缩放画布后导出，图片会变成空白或只剩图例，因为节点坐标被裁剪掉了。A 在测试中发现并和我沟通后，我修改了导出逻辑：忽略当前的缩放和平移，按所有节点重新计算图片尺寸，完整导出整张关系图。后来 A 又发现图例会遮挡右上角节点和自环，我通过为图例预留独立顶部空间，并让节点整体向下平移来解决遮挡的问题。经过这两轮修复，导出的 PNG 终于实现

了所有的要求。

4. 契约驱动开发与接口对齐

这次项目让我深刻体会到接口契约的重要性。在线上会议里面我们对齐了接口，还确定了 `setGraph`、`exportPNG` 等关键方法的方法名和参数约定，后续编码基本没有因为接口不一致而返工。我也学会了在修改公开方法时先和调用方沟通，比如新增 `zoomIn()`、`zoomOut()`、`getLayoutMode()` 时，都会在群里说明用途，确保 B 能正确调用。

5. Git 协作与版本管理

以前我用 Git 只会最简单的提交，这次多人协作让我熟悉了分支管理、Pull、Commit、Push、合并冲突处理等流程。我学会了在 `dev-c` 分支开发，定期拉取 `main` 和其他分支的最新代码，确保自己的修改不落后。也明白了本地改动不能随意提交，必须经过测试和确认后再推送。

三、团队合作中的体会

这次的五人小组项目合作，让我真正理解了“团队合作不只是把任务切完各做各的”，中间需要大量沟通、对齐和互相补位。

1. 与 A：接口对齐与 PNG 复核

A 负责算法和整体架构。在线上会议与群聊里，我和 A 确认了 `GraphPanel` 与 `model` 包之间的调用关系，包括 `getVertexNames`、`getSuccessors`、`getInDegree`、`getOutDegree` 等。后来 A 在测试中两次发现导出问题：第一次是平移后导出空白，第二次是图例遮挡节点。A 每次都给了非常具体的复现步骤和截图，让我能快速定位问题。我修复后，A 又重新测试确认通过。这个过程让我明白，从使用者角度检查成果有多重要，也让我更愿意认真对待别人对自己模块的反馈。

2. 与 B：可视化优化与集成修复

B 负责 GUI 主框架和交互控制，是和我的模块对接最多的人。她在集成测试后给我发了一份《可视化优化需求清单》，包括图例精简、节点序号徽章、边跟随拓扑序高亮、右下角缩放按钮、左上角放大查看等功能修改需求。我按清单逐项实现后，B 通过测试又发现放大查看窗口不显示环形布局这一问题。为解决这一问题，我新增了 `getLayoutMode()` 并在弹窗中同步布局模式。此外，在后续对接时 B 发现了鼠标滚轮缩放集成后失效的问

题。经过我们两人的讨论，B 将其改成工具栏按钮，我在 GraphPanel 中提供了 `zoomIn()` / `zoomOut()` 方法以实现按钮与鼠标滚轮可同时实现放大缩小的功能。这些来回修改让我体会到，集成阶段的问题往往不是单模块的 bug，而是模块之间的“缝隙”，需要双方一起排查。

3. 与 D：用户手册素材支持

D 负责数据文件和用户手册。在写 T-D8 用户手册时，D 需要我提供图片导出说明、布局切换说明和常见问题解答。我整理了一份详细的素材，包括 PNG 命名规则、中文字体处理、两种布局的适用场景，以及我在测试中遇到过的导出问题与修复情况。D 收到后直接用于用户手册，减少了反复沟通的成本。

4. 与 E：算法讲解稿与录屏支持

E 负责测试、工程交付与录制演示视频的任务。我给她提供了一份可视化模块的算法讲解稿，涵盖分层布局、环形布局、图绘制、交互和 PNG 导出等功能的算法讲解。E 在录视频前，我提醒她拉取修改好的最新的 dev-c 代码，确保演示的是修复后的版本。这种“交付+使用前说明”的配合方式，让我意识到文档和沟通本身也是开发工作的一部分。

5. 群聊与会议：并肩解决困难

项目期间我们开了 5 次线上会议，会对每个成员近期的工作进度进行同步，并确定下一阶段每个成员的目标。本次项目中的很多问题我们都是在群里通过文字、截图和不断地测试与反馈进行解决的。比如 A 发出导出 PNG 空白的截图后，我们在群里进行讨论，最终确定了忽略视图变换的方案；B 发现环形布局弹窗问题后，我很快在群里回复并修复。这些经历让我感受到，一个团队要高效运转，靠的是每个人主动沟通、及时反馈、互相补位，而不是各自埋头写代码。

四、不足与改进

回顾整个过程，我也有一些需要改进的地方。

1. 前期接口理解可以更深入

项目初期，我对 GraphPanel 与外部模块的交互边界理解不够细，导致集成后才发现滚轮缩放会被外层容器拦截。以后我会更早画出模块交互图，列出所有可能被外部影响的入口，提前和调用方对齐。

2. 测试覆盖不够全面

我主要靠手动测试和 `GraphPanel.main` 自测，没有为导出功能写自动化测试。像“平移后导出”、“缩放后导出”与“图例是否遮挡”这些问题，都是在集成阶段才被 A 和 B 发现。以后我会为关键路径补充小规模自动化测试，尤其是边界场景。

3. 文档整理可以更及时

我的很多验证记录、沟通结论散落在群聊和本地文件里，到最后才整理成 `docs/C/` 下的正式文档。如果每个阶段结束就及时归档，后面会轻松很多。

4. 时间安排可以更均匀

前期我花了很多时间在绘制细节上，后期修复导出和集成问题时比较赶。以后我会更早预留集成和回归测试的时间，避免最后阶段压力过大。

五、总结

这次高级算法原理实践项目，让我从一个只关心“算法能不能跑通”的学生，逐渐成长为能考虑调用者、使用者和协作成员的开发者。我完成了可视化模块的全部功能，修复了多个导出和集成问题，也和 A、B、D、E 一起经历了从需求、设计、编码到测试、交付的完整流程。

我特别感谢组长 B 在集成和可视化优化中的细致反馈，感谢 A 在接口对齐和 PNG 复核中的严格把关，感谢 D 在测试与用户手册素材上的认真对接配合，感谢 E 在测试和录屏中的支持。没有大家的互相配合，这个项目不可能顺利推进。

这次经历让我对团队协作、契约驱动开发和工程质量有了更深的理解，也让我对以后参与更复杂的软件项目更有信心。